

IFN680- Project 3

- Aircraft

Classification

The aircraft classification

In this challenge, you will act as a Machine Learning Engineer tasked with classifying images across 20 aircraft categories. You are required to follow a scientific workflow: establish a baseline, perform three controlled experiments to improve performance, and select your best-performing model for deployment. Your goal is **NOT** just to achieve high accuracy on your training data, but to build a model that generalises effectively to the unknown data. Work through the following tasks to complete this challenge:

Data

Download the following dataset for this challenge:

- [FGVCAircraft_Subset20.zip](#)
- [Download FGVCAircraft_Subset20.zip](#)
-

The dataset provided for this challenge is a curated subset of the FGVC-Aircraft collection. The dataset comprises thousands of images categorised into 20 distinct classes, labelled simply as Class_01 through Class_20. It is partitioned into two splits:

- **Trainval:** This is your sandbox. It contains 20 subfolders, each representing one class. You must further split this data into your own Training and Validation sets (e.g., an 80/20, 90/10, 10-fold split). Then, use this to train your weights and tune your hyperparameters. Your validation accuracy will be your only indicator of how well you might perform on the test set.
- **Test:** This data split SHOULD NOT be used for training or hyperparameter tuning. You will use this data set only to evaluate your final model. **As an evaluation metric, you should use the average per-class accuracy** as suggested by the dataset publication.

Task 1: Establish a baseline

The baseline is your 'starting point.' It should be a simple, working version of your classification pipeline that you can build quickly. Choose a standard architecture (e.g., ResNet-18) and train it with default settings and minimal preprocessing. Record your internal Validation Accuracy. This score represents the 'floor', every future experiment must aim to beat this number.

Task 2: Conduct 3 experiments

Now, try to improve your score by testing three different 'hypotheses.' Remember the rule of Controlled Experiments: change only one variable at a time so you know exactly what caused the improvement. You are free to perform any modifications you like, but we recommend focusing on these four areas:

- **Hyperparameter tuning:** Systematically adjust your learning rate, batch size, or weight decay. You can even use Optuna to automate this search as demonstrated in our tutorial.
- **NN architecture:** Switch your model's backbone. For example, swap a standard VGG for a ResNet, or experiment with more modern architectures (EfficientNet) and feature extractors (DinoV3).
- **Data augmentation:** Implement different data augmentation strategies available in PyTorch.
- **Optimisation:** Change how your model learns. This could involve trying different optimisers (e.g., swapping Adam for AdamW), implementing a Learning Rate Scheduler (like Cosine Annealing), or refining your fine-tuning strategy (e.g., freezing different layers of a pre-trained model).

Take Note: These hypotheses should be tested on your validation split, not on the test set.

Task 3: Select the best model

Analyse the results of your experiments and decide on your final configuration. Compare the Validation Accuracy of your experiments against the baseline. You may combine multiple successful features (e.g., your best Architecture + your best Augmentation) into a final 'Winning Model'. You should retrain the final model on the whole trainval. Subsequently, evaluate only your 'Winning Model' in the test set by reporting the average per-class accuracy. **We expect your best model to achieve an average per-class accuracy in the test split greater than 0.75 (a reasonable reference performance achieved in our experiments).** Additionally, you may utilise other plots and metrics for further analysis.

Submission instructions

Report

You will submit a concise PDF report (no more than 2 pages) documenting your development journey on this [page](#). You **must include** the following sections:

- **Baseline:** Describe your initial setup and its performance.
- **Experiments 1, 2, and 3:** For each, explain your Hypothesis ('I tried X because'), show the Learning Curves (plotted on a single graph for comparison), and report the accuracy in a comparison table.
- **Best model:** Identify which configuration or combination of configurations you chose as your final model and provide a brief rationale (e.g., best mean accuracy vs. stability vs best accuracy in the hardest class). Report the average per-class accuracy achieved by your best model in the test split as well any other metrics you computed. Additionally, discuss both the potential of your method for the target application and its anticipated limitations.

Project Code

You will submit a Zipped folder named **project3_code.zip** on this [page](#). This folder **must include**:

- development.ipynb: Jupyter Notebook used to train and validate your baseline, each hypothesis, and your final model. You can split these into

multiple files (like `development_baseline.ipynb`, `development_h1.ipynb`, ..) or a single file. Please follow the structure presented in the workshops (data loading, model definition, training loop, and evaluation). Ensure the notebook is commented and can run in the IFN680 computing environment.

- `main_report.ipynb`: Jupyter Notebook (.ipynb) that reproduces the learning curves, metrics computed and any other quantitative and qualitative evaluation for the baseline, three hypotheses and the best model described in your report. The grader will run this notebook to verify reproducibility. Please include any auxiliary files required for this process, such as pickle files containing losses, model weights or other necessary data. Ensure this notebook with these auxiliary files runs in the IFN680 computing environment.

Supporting resources

- [4.3 Tutorial](#) notebook: Please refer to this notebook for guidance on data loading, model setup, training, and evaluation loops using PyTorch. Your final notebook submission should adhere closely to the clean and organised structure presented in this workshop.
- [PyTorch documentation](#)
- [Links to an external site.](#)
- (PyTorch, n.d.): Use this as your primary reference for model layers, data augmentation, loss functions, and learning rate schedulers.
- Use the IFN680 GPU environment. Ensure your code is configured to utilise the available hardware via `.to(device)` to ensure your experiments run within a reasonable timeframe on the GPU.